



Training



Goals



Bugs

Strategy

Don't lose your cadence

COLOMBIA
KCD

GROWING CLOUD NATIVE TOGETHER

BOGOTÁ, 2025



@KCDCOLOMBIA

APOYADO POR



CLOUD NATIVE
COMPUTING FOUNDATION



Enhancing Stability and Enabling Cloud-Native Capabilities for Private Cellular Networks

Luis Felipe Ariza Vesga

Beanters | Kubestronaut-telco-cloud-AI



Agenda

1. Whoami.
2. Cloud-native project Lifecycle
3. General-purpose hardware and Open-source software for private cellular networks.
4. Initial use case: 4G private cellular network with SMS, data, VoLTE, and IPTV.
5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enabling CI/CD
6. Migrating from docker compose to Helm.
 1. Docker compose, Kompose, Helm, Kustomize
7. Management of external ports (USB3.0)
8. Security issues
9. Benchmark Docker compose vs Helm capabilities
10. Conclusions and takeaways



1. Whoami

Kubestronaut

Phd in telecommunications

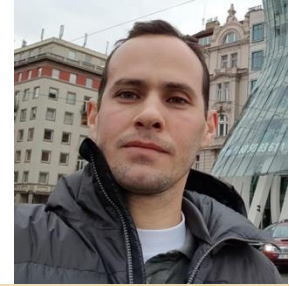
Master in electronics and computers

Electronic engineer

Colombia OpenInfra User group Organizer

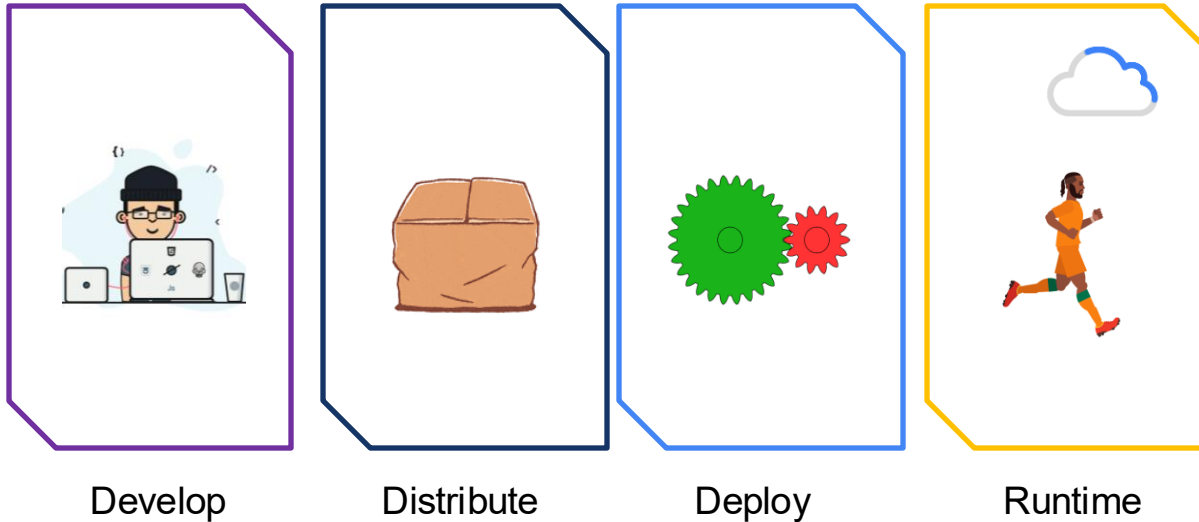
KCD Colombia 2025 co-organizer

Open-Source evangelist, research and development to connect traditionally unconnected areas

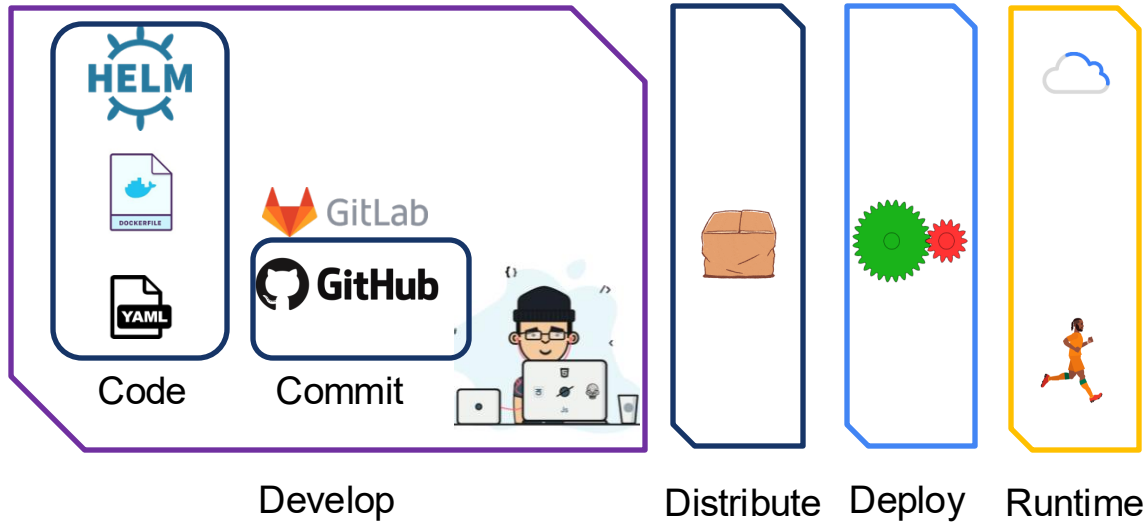


Infrastructure foundations for automatic cellular network deployments

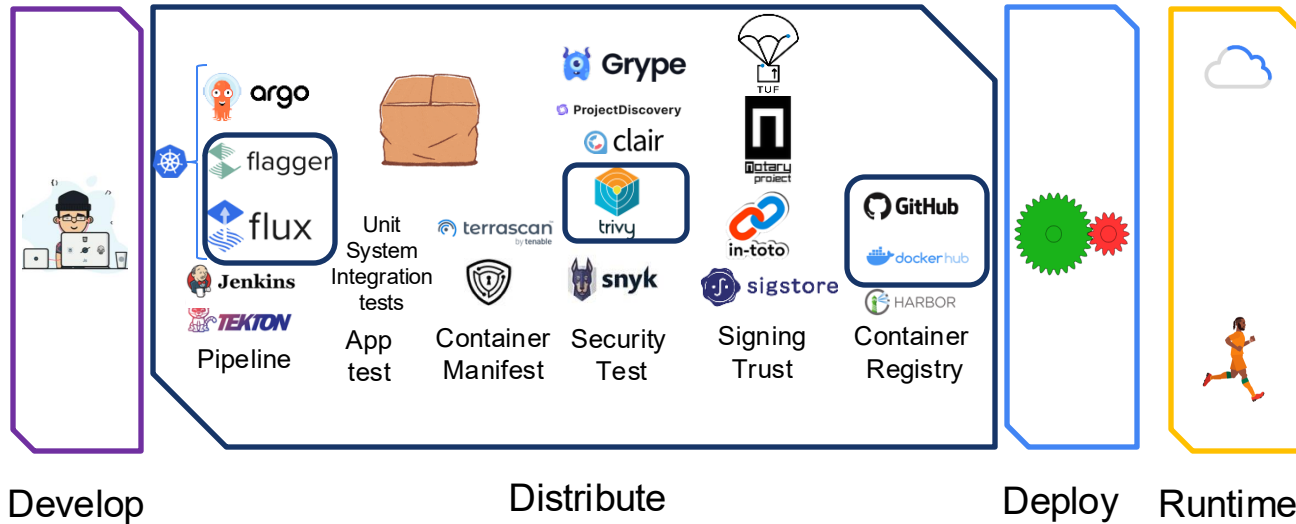
2. Cloud-native project Lifecycle



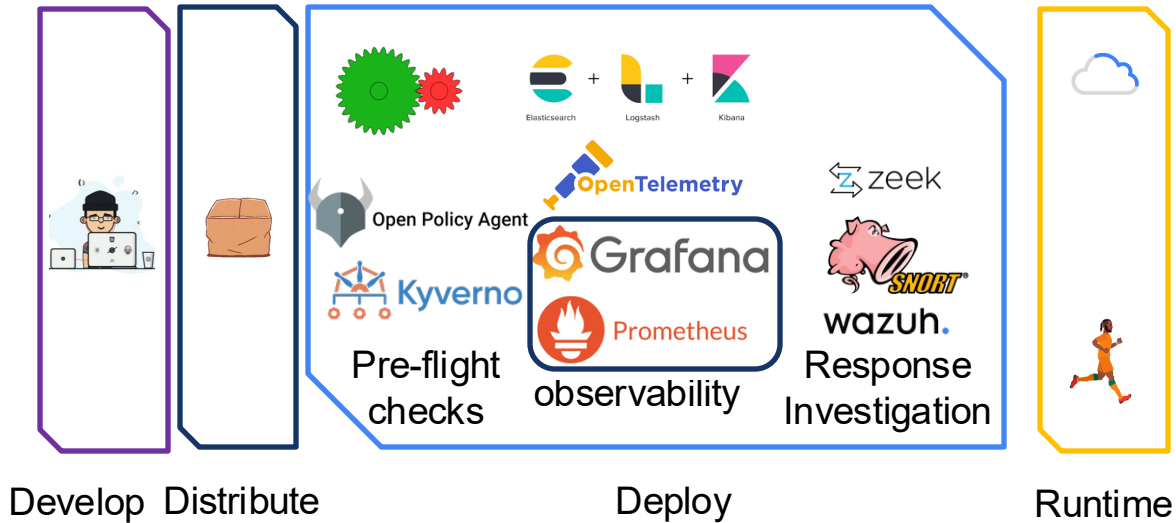
2. Cloud-native project Lifecycle.



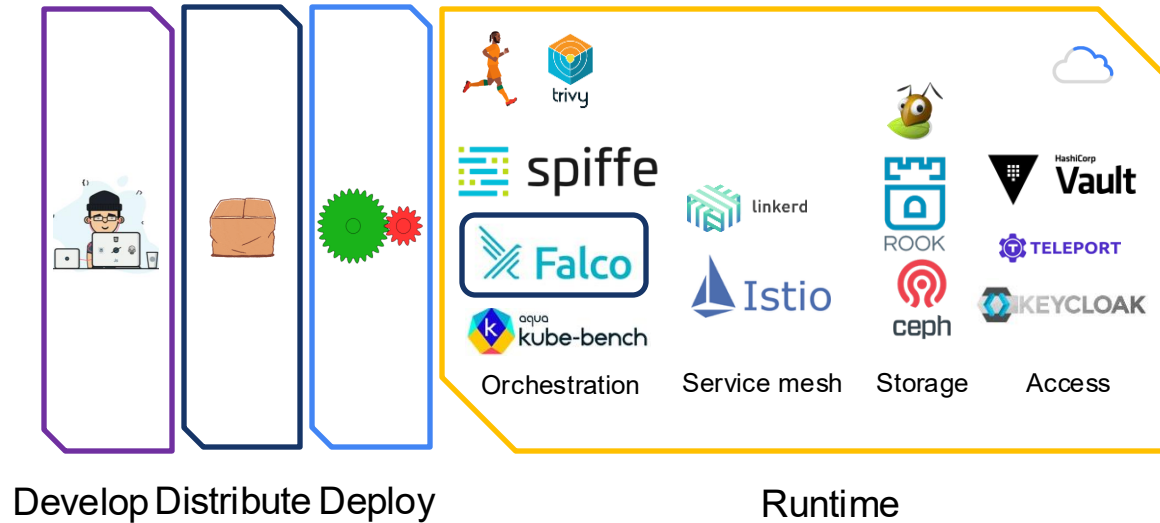
2. Cloud-native project Lifecycle



2. Cloud-native project Lifecycle



2. Cloud-native project Lifecycle



2. Cloud-native project Lifecycle



Easy to extend
Code is known
Complex configuration

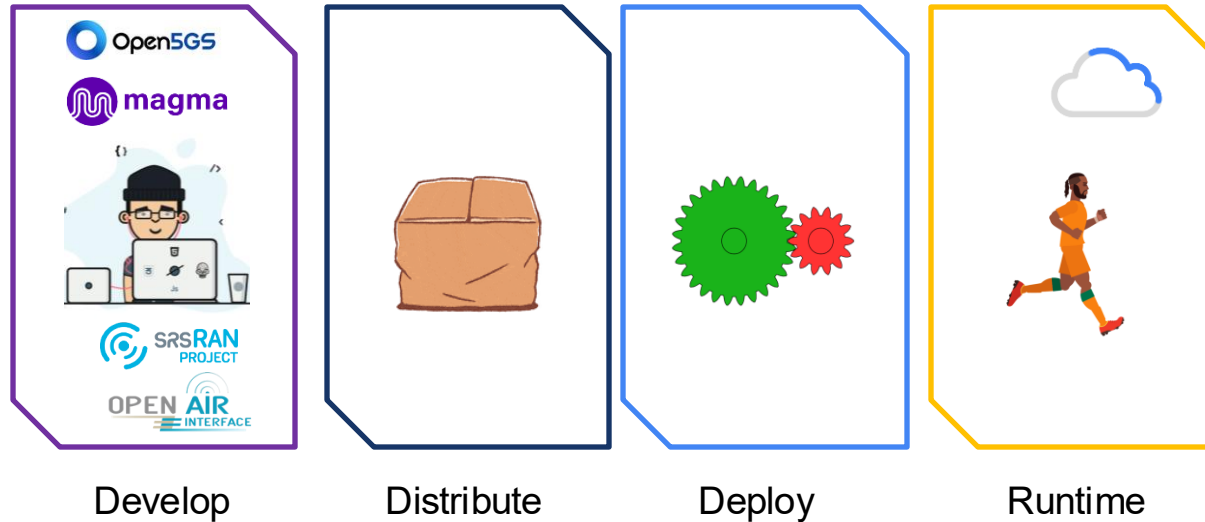
VS

Which one?
It depends

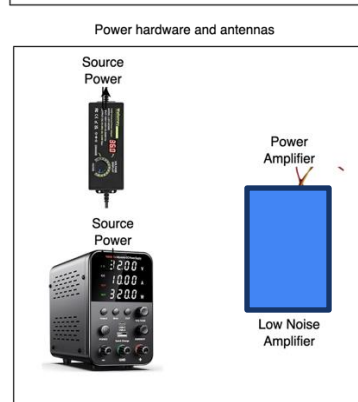
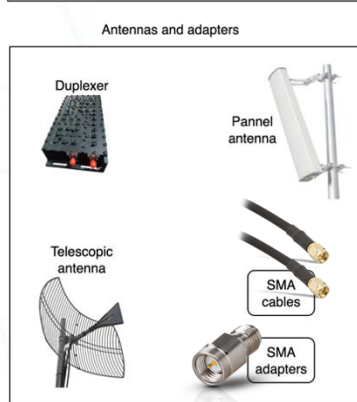
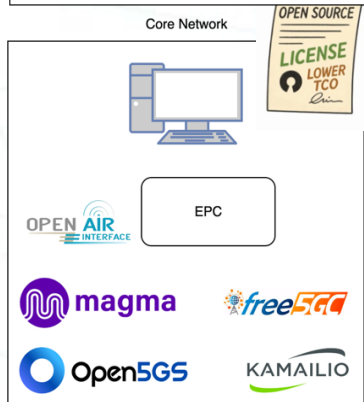
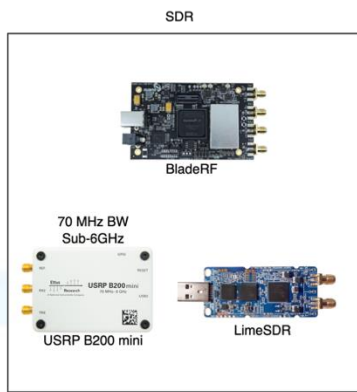
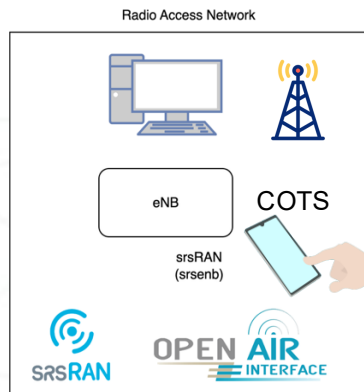


Easy to deploy
Code is unknown
Pay for upgrading

3. General-purpose hardware and Open-source software for private cellular networks.

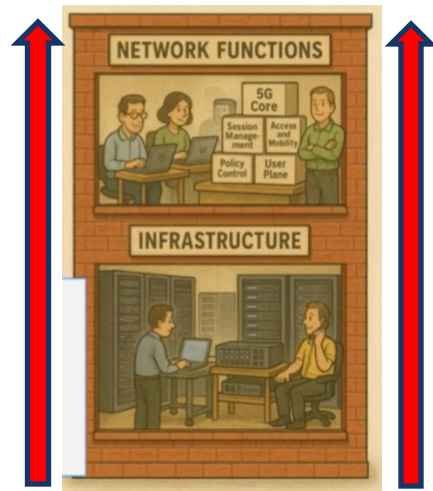
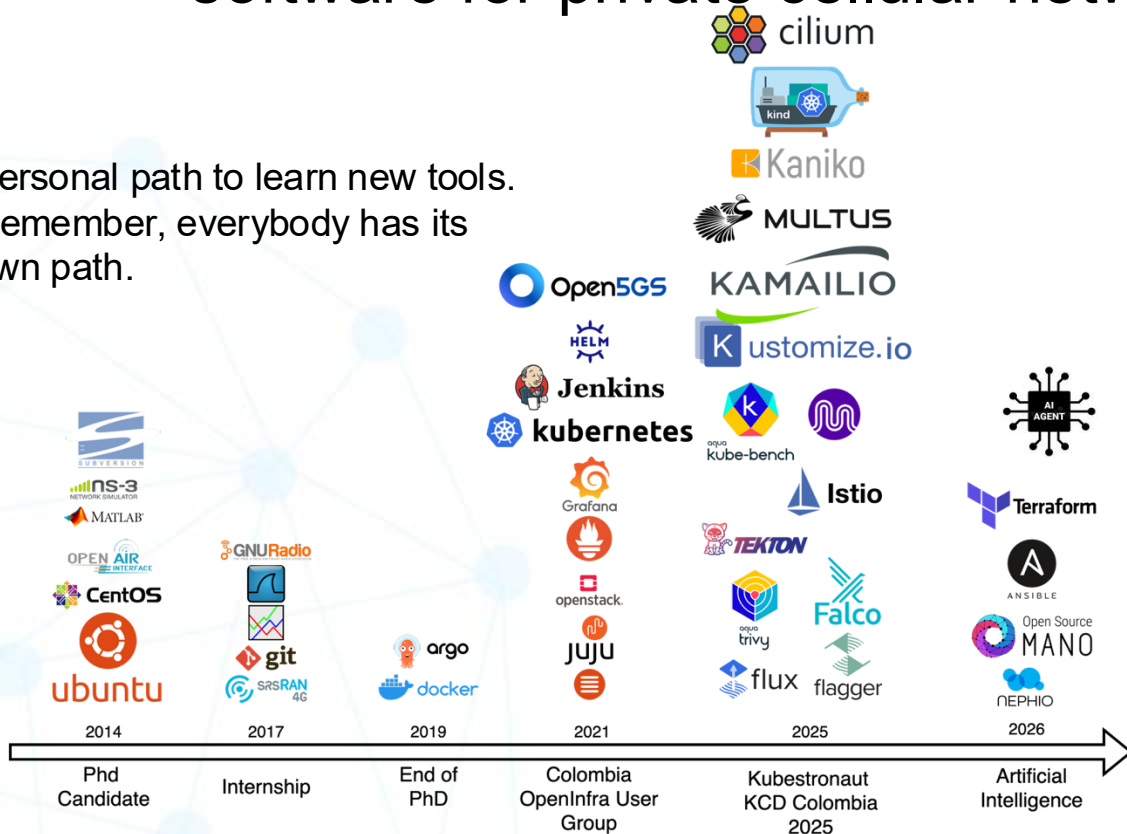


3. General-purpose hardware and Open-source software for private cellular networks.



3. General-purpose hardware and Open-source software for private cellular networks.

Personal path to learn new tools.
Remember, everybody has its own path.

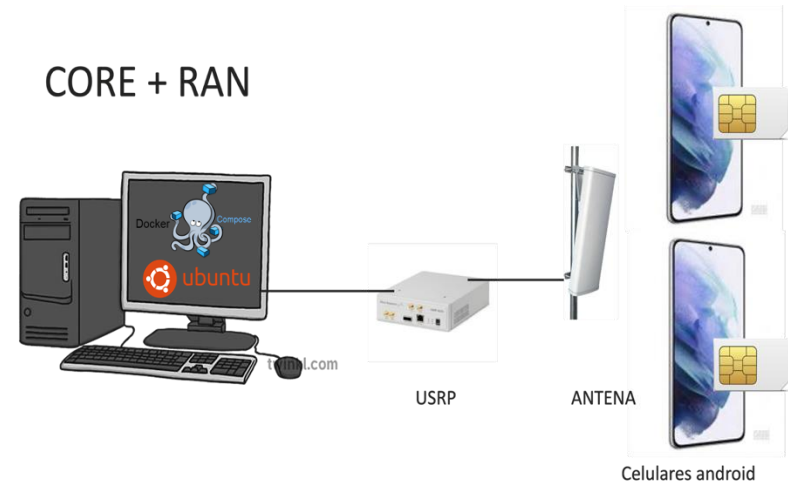


Infrastructure foundations for automatic cellular network deployments

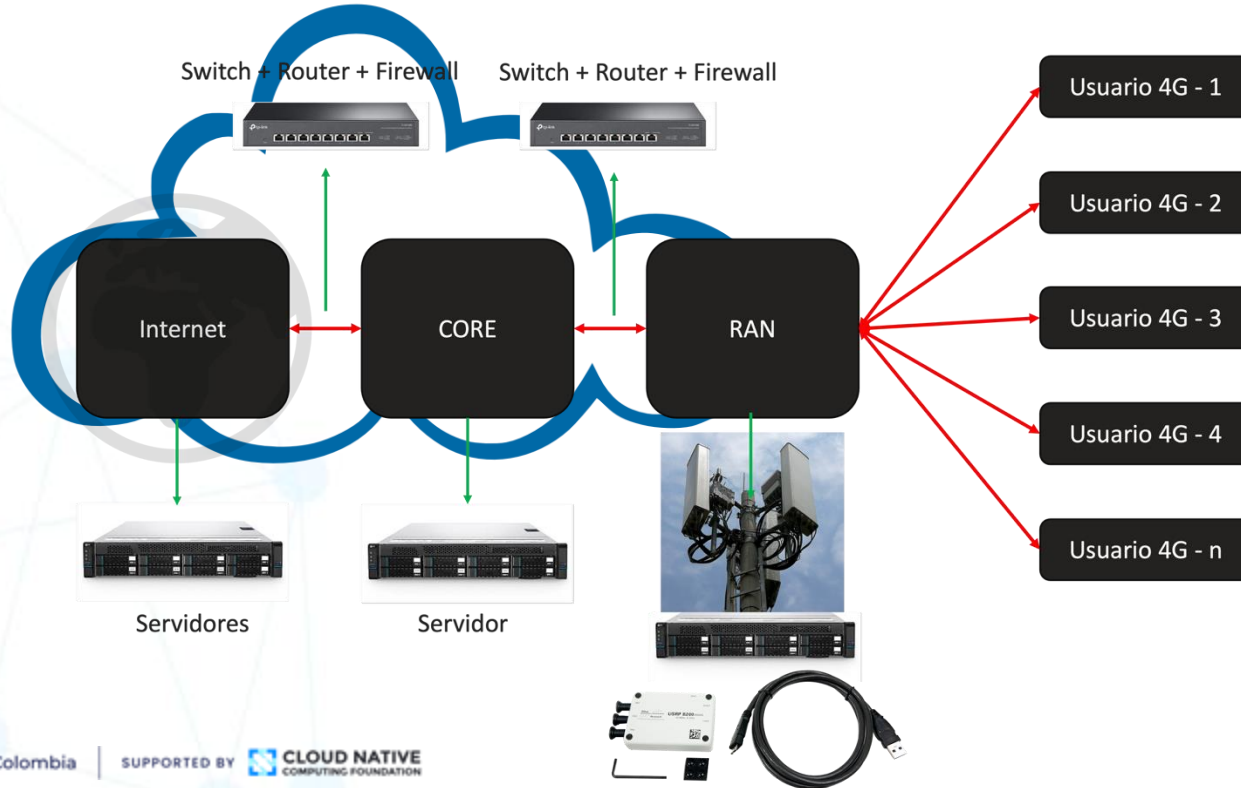
4. Use case: private cellular network with SMS, data, VoLTE, and IPTV.

Technical Specifications y/o limitations

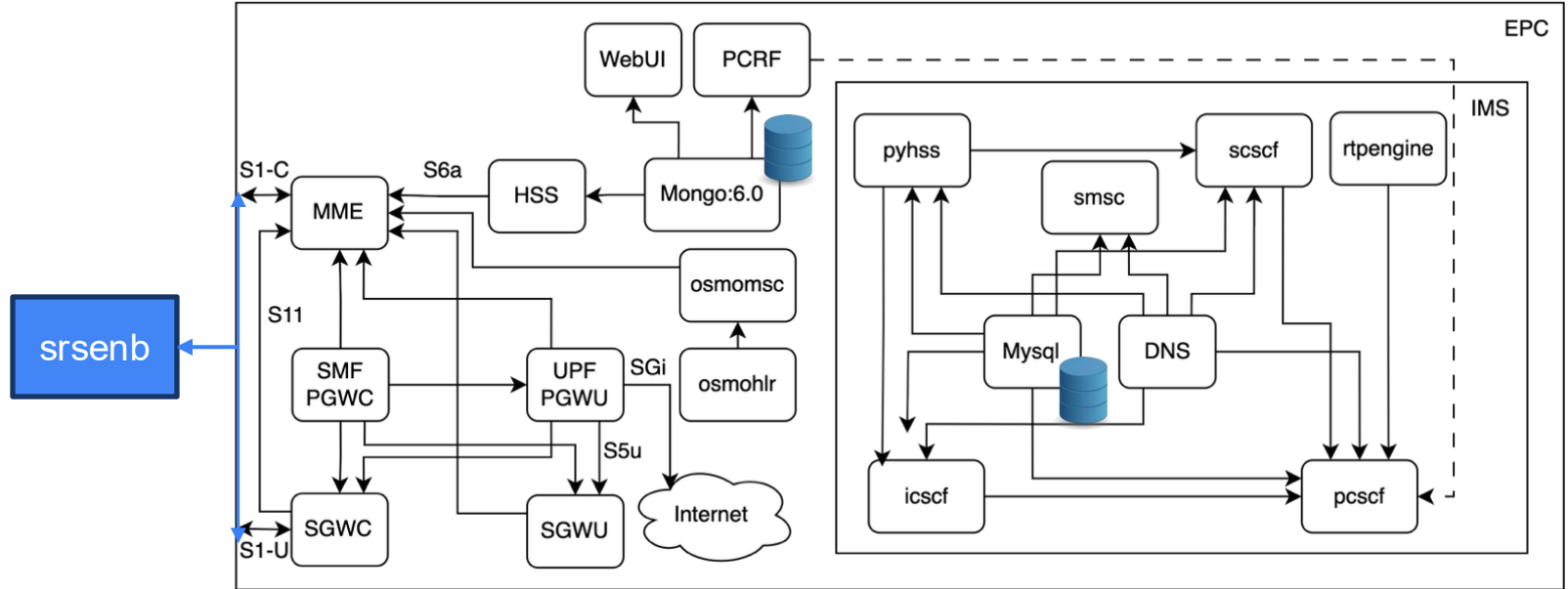
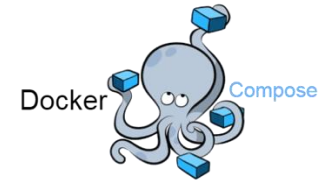
1. **Supported bandwidths:** 3 MHz, 5 MHz, 10 MHz
2. **Operating system:** Ubuntu 22.04
3. **SDRs tested:** USRP, LimeSDR
4. **Maximum number of subscribers:** 1024
5. **Maximum number of tested subscribers:** 3
6. **Tested bands:** B8 (900 MHz), B7 (2600 MHz)
7. **Duplex mode:** FDD
8. **External PA + LNA:** Band B8
9. **USB port version:** 3.0
10. **Antennas tested:** omnidirectional, panel, parabolic
11. **Tested SIM cards:** Sysmocom, OpenCells



4. Use case: private cellular network with SMS, data, VoLTE, and IPTV.

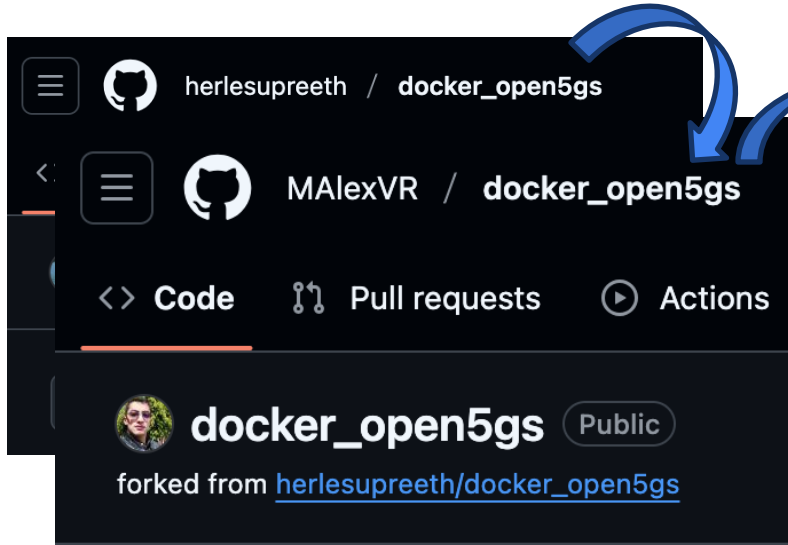


4. DEMO



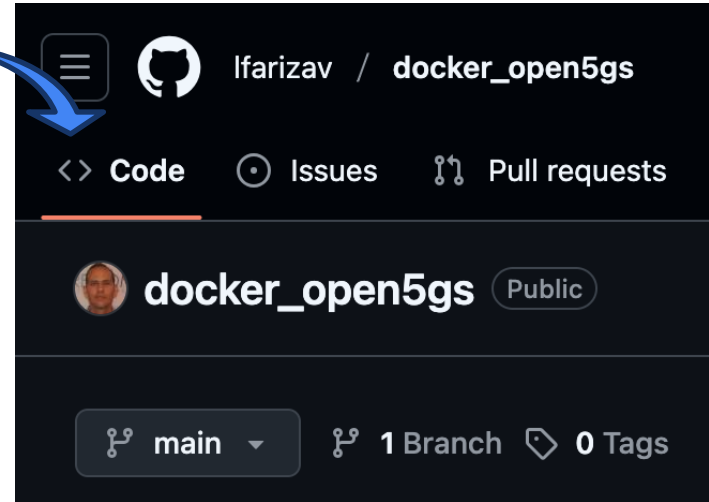
Containers architecture for data and VoLTE (IMS) services

4. DEMO

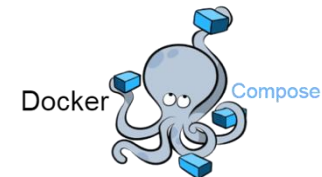


https://github.com/herlesupreeth/docker_open5gs

https://github.com/MAlexVR/docker_open5gs



https://github.com/lfarizav/docker_open5gs



4. DEMO

```
luis@lfarizav:~$ cd docker_openSgs
luis@lfarizav:~/docker_openSgs$ set -a
source .env
sudo ufw disable
sudo systemctl -w net.ipv4.ip_forward=1
sudo cpupower frequency-set -g performance
Firewall stopped and disabled on system startup
net.ipv4.ip_forward = 1
Setting cpu: 0
Setting cpu: 1
Setting cpu: 2
Setting cpu: 3
Setting cpu: 4
Setting cpu: 5
Setting cpu: 6
Setting cpu: 7
Setting cpu: 8
Setting cpu: 9
Setting cpu: 10
Setting cpu: 11
luis@lfarizav:~/docker_openSgs$ docker compose -f docker-compose.yaml -d
unknown shorthand flag: 'd' in -d
luis@lfarizav:~/docker_openSgs$ docker compose -f docker-compose.yaml up -d
[+] Running 26/26
✔ Network docker_openSgs_default Created
✔ Container mongo Started
✔ Container promtail Started
✔ Container snf Started
✔ Container rtpengine Started
✔ Container mysql Started
✔ Container portainer Started
✔ Container loki Started
✔ Container metrics Started
✔ Container dns Started
✔ Container osmohlr Started
✔ Container srsenb Started
✔ Container grafana Started
✔ Container osmoms Started
✔ Container webui Started
✔ Container pyhss Started
✔ Container hss Started
✔ Container pcrf Started
✔ Container smsc Started
✔ Container upf Started
✔ Container sgwc Started
✔ Container sgwu Started
✔ Container scscf Started
✔ Container icscf Started
✔ Container mme Started
✔ Container pcscf Started
luis@lfarizav:~/docker_openSgs$
```

Open5GS

Subscriber

Profile

Account

001010000010001

001010000010002

001010000010003

001010000010004

001010000010005

001010000010006

001010000010007

001010000010008

001010000010009

001010000010010

001010000010011

001010000010012

001010000010013

001010000010014

001010000010015

001010000010016

001010000010017

001010000010018

001010000010019

001010000010020

001010123456790

Upgrade to Business Edition

portainer.io

COMMUNITY EDITION

Home

local

Dashboard

Templates

Stacks

Containers

Images

Networks

Volumes

Events

Host

Administration

User-related

Containers

Container list

Containers

Search...

Start

Stop

Kill

Restart

Pause

Resume

Remove

Add container

Containers

Table with 9 columns: Name, State, Quick Actions, Stack, Image, Created, IP Address, Published Ports, Ownership

4. DEMO

##4636####*: secret android code

Test Site - pyhss - PyHSS OAM API ^{1.0}
[Base URL: /]
Swagger JSON

Restful API for working with PyHSS

apn PyHSS APN Functions

auc PyHSS AUC Functions

subscriber PyHSS SUBSCRIBER Functions

ims_subscriber PyHSS IMS SUBSCRIBER Functions

PUT /ims_subscriber/ Create new IMS SUBSCRIBER

GET /ims_subscriber/ims_subscriber_imsi/{imsi} Get IMS data for imsi

GET /ims_subscriber/ims_subscriber_msisd/{msisd} Get IMS data for MSISDN

GET /ims_subscriber/List Get all IMS Subscribers

DELETE /ims_subscriber/{ims_subscriber_id} Delete all data for specified ims_subscriber_id

PATCH /ims_subscriber/{ims_subscriber_id} Update IMS SUBSCRIBER data for specified ims_subscriber

GET /ims_subscriber/{ims_subscriber_id} Get all SUBSCRIBER data for specified ims_subscriber_id

Open5GS | Claro

Fri, Jul 25

Phone info1

Phone 0

IMEI: 869536071370822

Phone Number: 10008 { CARRIER, UICC:10008, IMS: }

Current subid: 8

Subid of default data SIM: 4

IMSI: 001010000010008

Current Network: Open5GS

Roaming: Not Roaming

Data Service: Connected

Data Network Type: IWLAN

Override Network Type: NONE

Voice Service: In Service

Voice Network Type: LTE

Signal Strength: -105 dBm 35 asu

DL Bandwidth (kbps): 13183

UL Bandwidth (kbps): 13183

EN-DC Available (NSA): false

DCNR Restricted (NSA): false

NR Available (NSA): false

NR State (NSA): NONE

NR Frequency: UNKNOWN

Network Slicing Config: Unable to get slicing config.

Physical Channel Configurations: {}

Set Preferred Network Type: NR/LTE

Mobile Radio Power

VoLTE Provisioned

Video Calling Provisioned

Wifi Calling Provisioned

EAB/Presence Provisioned

Enable DSDS

Set Removable eSIM as Default

RUN PING TEST

Ping Hostname(www.google.com) IPv4:
Ping Hostname(www.google.com) IPv6:
HTTP Client Test:
Data Sent: 920 pkts, 162967 bytes
Data Received: 663 pkts, 305430 bytes
Number of PPP Reset Since Boot:
Voice Call Status: Idle
Message Waiting: false
Cell Redirect: false
Cell Info Refresh Rate: Immediate

All Cell Measurement Info:

LTE	SRV	MCC	MNC	TAC	CID	PCI	EARFCN	BSW	RSRP	RSRQ
TA	R+N	1	1	1	105217	1	3150	10	-92	-3
0										

CARRIER PROVISIONING INFO

TRIGGER CARRIER PROVISIONING

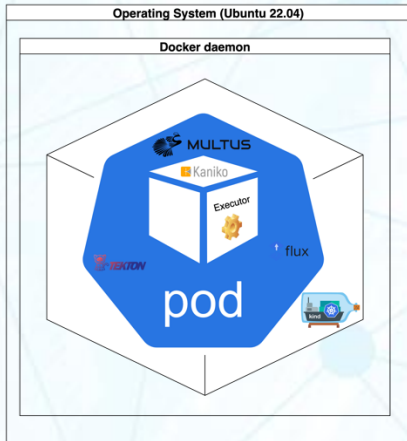
OEM-SPECIFIC INFO/SETTINGS

SMSC: UPDATE REFRESH

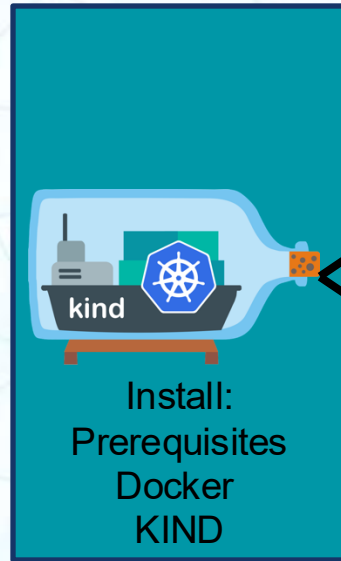
TOGGLE DNS CHECK: 0.0.0.0 not allowed

5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enabling CI/CD

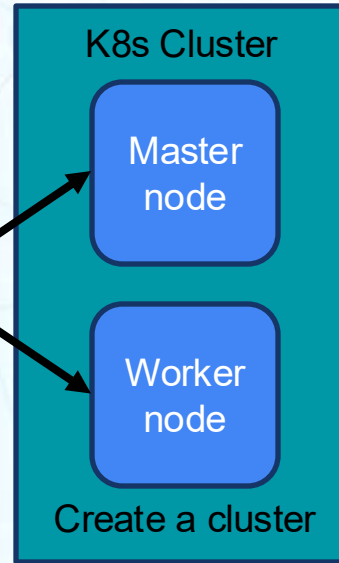
Virtualization levels



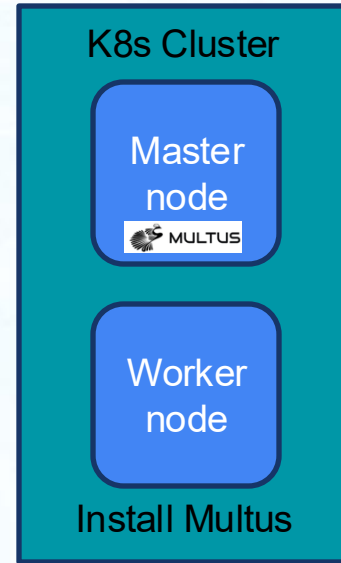
Step 1:



Step 2:



Step 3:



K8s Cluster
1 controlplane
1 node (worker)



Storage issue
temporary

5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enabling CI/CD

Docker:

prerequisites

```
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Helm:

```
curl https://baltocdn.com/helm/signing.asc | gpg --dearmor | sudo tee /usr/share/keyrings/helm.gpg > /dev/null
sudo apt-get install apt-transport-https --yes echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/helm.gpg] https://baltocdn.com/helm/stable/debian/ all main" | sudo tee
/etc/apt/sources.list.d/helm-stable-debian.list sudo apt-get update sudo apt-get install helm
```

KIND:

```
[ $(uname -m) = x86_64 ] && curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.29.0/kind-linux-amd64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind
```

5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enabling CI/CD

```
luis@lfarizav:~/setup/kind/kind-two-nodes-cluster$ ./install.sh -I
Install kind cluster called dev and Multus (CNI)
Creating cluster "dev" ...
  ✓ Ensuring node image (kindest/node:v1.32.0) 
  ✓ Preparing nodes  
  ✓ Writing configuration 
  ✓ Starting control-plane 
  ✓ Installing CNI 
  ✓ Installing StorageClass 
  ✓ Joining worker nodes 
Set kubectl context to "kind-dev"
You can now use your cluster with:

kubectl cluster-info --context kind-dev
```

Step 2: Create a cluster

```

luis@lfarizav:~/setup/kind/kind-two-nodes-cluster$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
dev-control-plane   NotReady control-plane  19s   v1.32.0
dev-worker          NotReady <none>      9s    v1.32.0

luis@lfarizav:~/setup/kind/kind-two-nodes-cluster$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
dev-control-plane   Ready     control-plane  24s   v1.32.0
dev-worker          Ready     <none>      14s   v1.32.0

```

Step 1: Install KIND

```

luis@lfarizav:~/setup/kind/two-nodes-cluster$ ./install.sh -M
Install Multus (CNI)
customresourcedefinition.k8s.io/network-attachment-definitions.k8s.cni.cncf.io created
clusterrole.rbac.authorization.k8s.io/multus created
clusterrolebinding.rbac.authorization.k8s.io/multus created
serviceaccount/multus created
configmap/multus-cni-config created
daemonset.apps/kube-multus-ds created
net.ipv4.ip_forward = 1
  % Total      % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left    Speed
  0     0    0     0    0    0      0      0  --:--:--  0:00:01  --:--:--    0
100 53.3M 100 53.3M    0    0 1910k      0  0:00:28  0:00:28  --:--:-- 1857k
net.ipv4.ip_forward = 1
  % Total      % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left    Speed
  0     0    0     0    0    0      0      0  --:--:--  0:00:01  --:--:--    0
100 53.3M 100 53.3M    0    0 1911k      0  0:00:28  0:00:28  --:--:-- 7636k
Multus was installed successfully on kubernetes

```

Step 3: Install Multus

5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enabling CI/CD

Step 3: Install Multus (Verify)

```
luis@lfarizav:~/setup/kind/kind-two-nodes-cluster$ kubectl get all -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
pod/coredns-668d6bf9bc-5lgf8	1/1	Running	0	2m6s
pod/coredns-668d6bf9bc-ph7ns	1/1	Running	0	2m6s
pod/etcd-dev-control-plane	1/1	Running	0	2m14s
pod/kindnet-5qsrw	1/1	Running	0	2m6s
pod/kindnet-pwk4x	1/1	Running	0	2m6s
pod/kube-apiserver-dev-control-plane	1/1	Running	0	2m14s
pod/kube-controller-manager-dev-control-plane	1/1	Running	0	2m13s
pod/kube-multus-ds-45fmf	1/1	Running	0	107s
pod/kube-multus-ds-jtf9k	1/1	Running	0	108s
pod/kube-proxy-pd7cl	1/1	Running	0	2m6s
pod/kube-proxy-zt4zh	1/1	Running	0	2m6s
pod/kube-scheduler-dev-control-plane	1/1	Running	0	2m14s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kube-dns	ClusterIP	10.96.0.10	<none>	53/UDP,53/TCP,9153/TCP	2m13s

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE SELECTOR	AGE
daemonset.apps/kindnet	2	2	2	2	2	kubernetes.io/os=linux	2m12s
daemonset.apps/kube-multus-ds	2	2	2	2	2	<none>	108s
daemonset.apps/kube-proxy	2	2	2	2	2	kubernetes.io/os=linux	2m13s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/coredns	2/2	2	2	2m13s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/coredns-668d6bf9bc	2	2	2	2m6s

5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enabling CI/CD

```
luis@lfarizav:~$ curl -s https://fluxcd.io/install.sh | sudo bash
[INFO] Downloading metadata https://api.github.com/repos/fluxcd/flux2/releases/latest
[INFO] Using 2.6.4 as release
[INFO] Downloading hash https://github.com/fluxcd/flux2/releases/download/v2.6.4/flux_2.6.4_checksums.txt
[INFO] Downloading binary https://github.com/fluxcd/flux2/releases/download/v2.6.4/flux_2.6.4_linux_amd64.tar.gz
[INFO] Verifying binary download
[INFO] Installing flux to /usr/local/bin/flux
luis@lfarizav:~$ which flux
/usr/local/bin/flux
```

Step 4:
Install
Flux and verify

```
luis@lfarizav:~$ k get all -n flux-system
```

NAME	READY	STATUS	RESTARTS	AGE
pod/helm-controller-75f5c877f4-d6gk7	1/1	Running	0	2m17s
pod/kustomize-controller-7c8f87b7fd-wxgr6	1/1	Running	0	2m17s
pod/notification-controller-84f6f496b5-dcgwx	1/1	Running	0	2m17s
pod/source-controller-5789bb5499-7ctlk	1/1	Running	0	2m17s

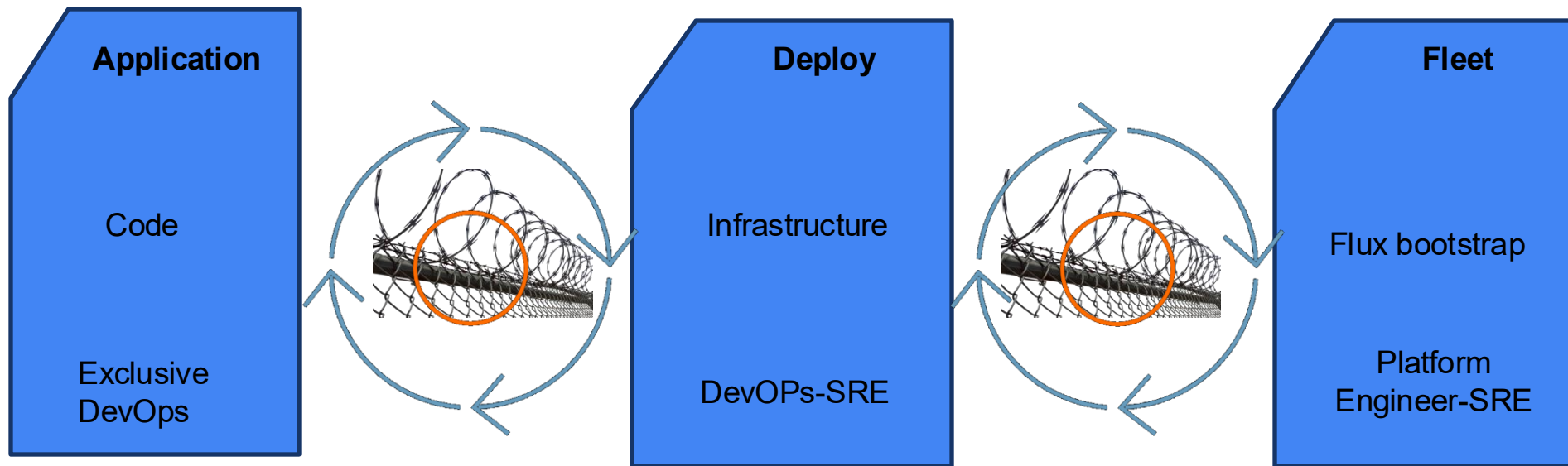
NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/notification-controller	ClusterIP	10.96.176.5	<none>	80/TCP	2m17s
service/source-controller	ClusterIP	10.96.59.121	<none>	80/TCP	2m17s
service/webhook-receiver	ClusterIP	10.96.168.167	<none>	80/TCP	2m17s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/helm-controller	1/1	1	1	2m17s
deployment.apps/kustomize-controller	1/1	1	1	2m17s
deployment.apps/notification-controller	1/1	1	1	2m17s
deployment.apps/source-controller	1/1	1	1	2m17s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/helm-controller-75f5c877f4	1	1	1	2m17s
replicaset.apps/kustomize-controller-7c8f87b7fd	1	1	1	2m17s
replicaset.apps/notification-controller-84f6f496b5	1	1	1	2m17s
replicaset.apps/source-controller-5789bb5499	1	1	1	2m17s

5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enabling CI/CD

Multitenancy

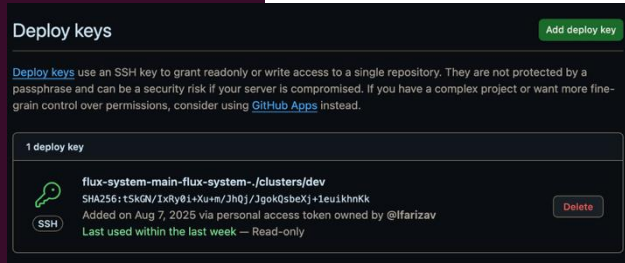


<https://github.com/fluxcd/flux2-multi-tenancy>

5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enabling CI/CD

```
luis@lfarizav:~$ flux check
▶ checking prerequisites
✓ Kubernetes 1.32.0 >=1.31.0-0
▶ checking version in cluster
✗ checking failed: customresourcedefinitions.apiextensions.k8s.io "gitrepositories.source.toolkit.fluxcd.io" not found
▶ checking controllers
✗ no controllers found in the 'flux-system' namespace with the label selector 'app.kubernetes.io/part-of=flux'
▶ checking crds
✗ no crds found with the label selector 'app.kubernetes.io/part-of=flux'
✗ check failed
luis@lfarizav:~$ flux bootstrap github --owner=$GITHUB_USER \
> --repository=k8s_open5gs \
> --branch=main-sms-data \
> --path=./clusters/dev \
> --personal \
> --log-level=debug \
> --network-policy=false \
> --components=source-controller,kustomize-controller,helm-controller,notification-controller
▶ connecting to github.com
▶ cloning branch "main-sms-data" from Git repository "https://github.com/lfarizav/k8s_open5gs.git"
✓ cloned repository
▶ generating component manifests
✓ generated component manifests
✓ committed component manifests to "main-sms-data" ("d65f263b04693dc33f29734c754ddf13f90a645b")
▶ pushing component manifests to "https://github.com/lfarizav/k8s_open5gs.git"
▶ installing components in "flux-system" namespace
✓ installed components
✓ reconciled components
▶ determining if source secret "flux-system/flux-system" exists
▶ generating source secret
✓ public key: ecdsa-sha2-nistp384 AAAAE2VjZHNhLXNoYTItbmlzdHAzODQAAAIbmlzdHAzODQAAABhBPBQ6BV+Lx4CEB7WQnZHfKYsTXziZKSLaErDhS
r7nTrMIHmTsw==
✓ configured deploy key "flux-system-main-sms-data-flux-system-./clusters/dev" for "https://github.com/lfarizav/k8s_open5gs"
▶ applying source secret "flux-system/flux-system"
✓ reconciled source secret
▶ generating sync manifests
✓ generated sync manifests
✓ committed sync manifests to "main-sms-data" ("fa7a1bf6cfc95ec3a5b7684462ff132a0bb0f92f")
▶ pushing sync manifests to "https://github.com/lfarizav/k8s_open5gs.git"
▶ applying sync manifests
✓ reconciled sync configuration
◎ waiting for GitRepository "flux-system/flux-system" to be reconciled
✓ GitRepository reconciled successfully
◎ waiting for Kustomization "flux-system/flux-system" to be reconciled
✓ Kustomization reconciled successfully
▶ confirming components are healthy
✓ helm-controller: deployment ready
✓ kustomize-controller: deployment ready
✓ notification-controller: deployment ready
✓ source-controller: deployment ready
✓ all components are healthy
```

Step 5:
Bootstrap
Flux:ssh key generation



5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enabling CI/CD

```
luis@lfarizav:~/k8s_open5gs$ flux create source git k8s-open5gs --url=https://github.com/lfarizav/k8s_open5gs --branch=main-sms-data --username=lfarizav --password=ghp_GnFn0uF1LGv1LSRFvVZQ
+ generating GitRepository source
+ applying secret with repository credentials
✓ authentication configured
+ applying GitRepository source
✓ GitRepository source created
@ waiting for GitRepository source reconciliation
✓ GitRepository source reconciliation completed
✓ fetched revision: main-sms-data@sha1:c80fb4f23842b10364bfa836f7e6004a818b1551
```

Step 6:
Create
source git
Create
helmrelease

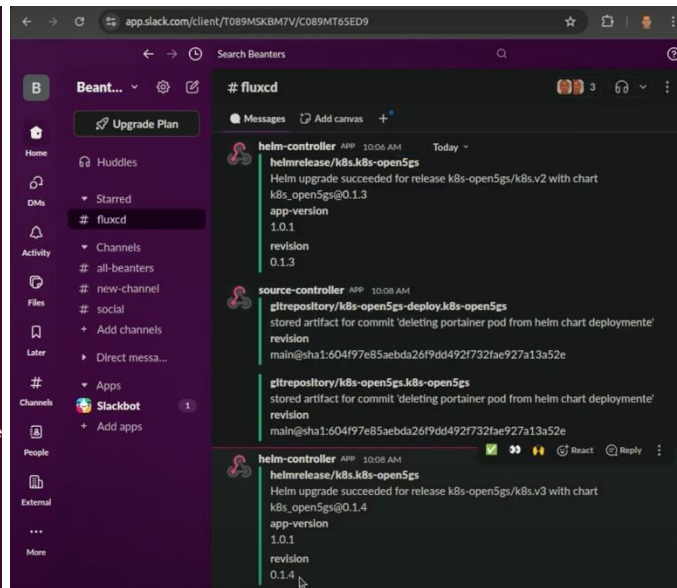
```
luis@lfarizav:~/k8s_open5gs$ flux create helmrelease k8s --interval=10m --source=GitRepository/k8s-open5gs --chart=./helm/k8s-open5gs
+ generating HelmRelease
+ applying HelmRelease
✓ HelmRelease created
@ waiting for HelmRelease reconciliation
✓ HelmRelease k8s is ready
✓ applied revision 0.1.0
```

5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enabling CI/CD

Step 7: Slack notifications

```
luis@lfarizav:~$ flux create alert slack-notif \
--provider-ref=slack \
--event-source=GitRepository/* \
--event-source=Kustomization/* \
--event-source=HelmRelease/* \
--event-severity=info \
--export
---
apiVersion: notification.toolkit.fluxcd.io/v1beta3
kind: Alert
metadata:
  name: slack-notif
  namespace: flux-system
spec:
  eventSeverity: info
  eventSources:
  - kind: GitRepository
    name: '*'
  - kind: Kustomization
    name: '*'
  - kind: HelmRelease
    name: '*'
  providerRef:
    name: slack
luis@lfarizav:~$ flux create alert slack-notif --provider-ref=slack
+generating Alert
+applying Alert
+Alert created
+waiting for Alert reconciliation
+Alert slack-notif is ready
luis@lfarizav:~$ flux get alert
NAME      SUSPENDED  READY  MESSAGE
slack-notif  False      True   Alert is Ready
```

```
luis@lfarizav:~$ flux get alert-providers
X no Provider objects found in "flux-system" namespace
luis@lfarizav:~$ flux create alert-provider slack \
--type=slack \
+generating Provider
+applying Provider
+Provider created
+waiting for Provider reconciliation
+Provider slack is ready
luis@lfarizav:~$ flux create alert-provider slack \
--type=slack \
--channel=fluxcd \
--secret-ref=slack-url --export
---
apiVersion: notification.toolkit.fluxcd.io/v1beta3
kind: Provider
metadata:
  name: slack
  namespace: flux-system
spec:
  secretRef:
    name: slack-url
    type: slack
luis@lfarizav:~$ flux create alert-provider slack --type=slack
+generating Provider
+applying Provider
+Provider updated
+waiting for Provider reconciliation
+Provider slack is ready
luis@lfarizav:~$ flux get alert-providers
NAME  READY  MESSAGE
slack True   Provider is Ready
```



5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enabling CI/CD

Step 4: Tekton (continuous integration)

```
luis@lfarizav:~/open5gs-infra/clusters/dev$ kubectl apply --filename https://storage.googleapis.com/tekton-releases/pipeline/latest/release.yaml
namespace/tekton-pipelines created
clusterrole.rbac.authorization.k8s.io/tekton-pipelines-controller-cluster-access created
clusterrole.rbac.authorization.k8s.io/tekton-pipelines-controller-tenant-access created
clusterrole.rbac.authorization.k8s.io/tekton-pipelines-webhook-cluster-access created
clusterrole.rbac.authorization.k8s.io/tekton-events-controller-cluster-access created
role.rbac.authorization.k8s.io/tekton-pipelines-controller created
role.rbac.authorization.k8s.io/tekton-pipelines-webhook created
role.rbac.authorization.k8s.io/tekton-pipelines-events-controller created
role.rbac.authorization.k8s.io/tekton-pipelines-leader-election created
role.rbac.authorization.k8s.io/tekton-pipelines-info created
serviceaccount/tekton-pipelines-controller created
serviceaccount/tekton-pipelines-webhook created
serviceaccount/tekton-events-controller created
```

```
luis@lfarizav:~$ # Get the tar.xz
curl -LO https://github.com/tektoncd/cli/releases/download/v0.41.0/tkn_0.41.0_Linux_x86_64.tar.gz
# Extract tkn to your PATH (e.g. /usr/local/bin)
sudo tar xvfz tkn_0.41.0_Linux_x86_64.tar.gz -C /usr/local/bin/ tkn
% Total    % Received % Xferd  Average Speed   Time    Time     Current
           Dload  Upload   Total   Spent    Left     Speed
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
100 34.6M 100 34.6M  0  0  9135k  0  0:00:03  0:00:03 --:--:-- 17.9M
tkn
luis@lfarizav:~$ tkn
CLI for tekton pipelines

Usage:
tkn [flags]
tkn [command]
```

```
Use "tkn [command] --help" for more information about a command.
luis@lfarizav:~$ tkn version
Client version: 0.41.0
Pipeline version: v1.3.0
```

5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enabling CI/CD

```
luis@lfarizav:~$ tkn hub install task git-clone
WARN: This version has been deprecated
Task git-clone(0.9) installed in default namespace
luis@lfarizav:~$ tkn hub install task kaniko
WARN: This version has been deprecated
Task kaniko(0.7) installed in default namespace
```

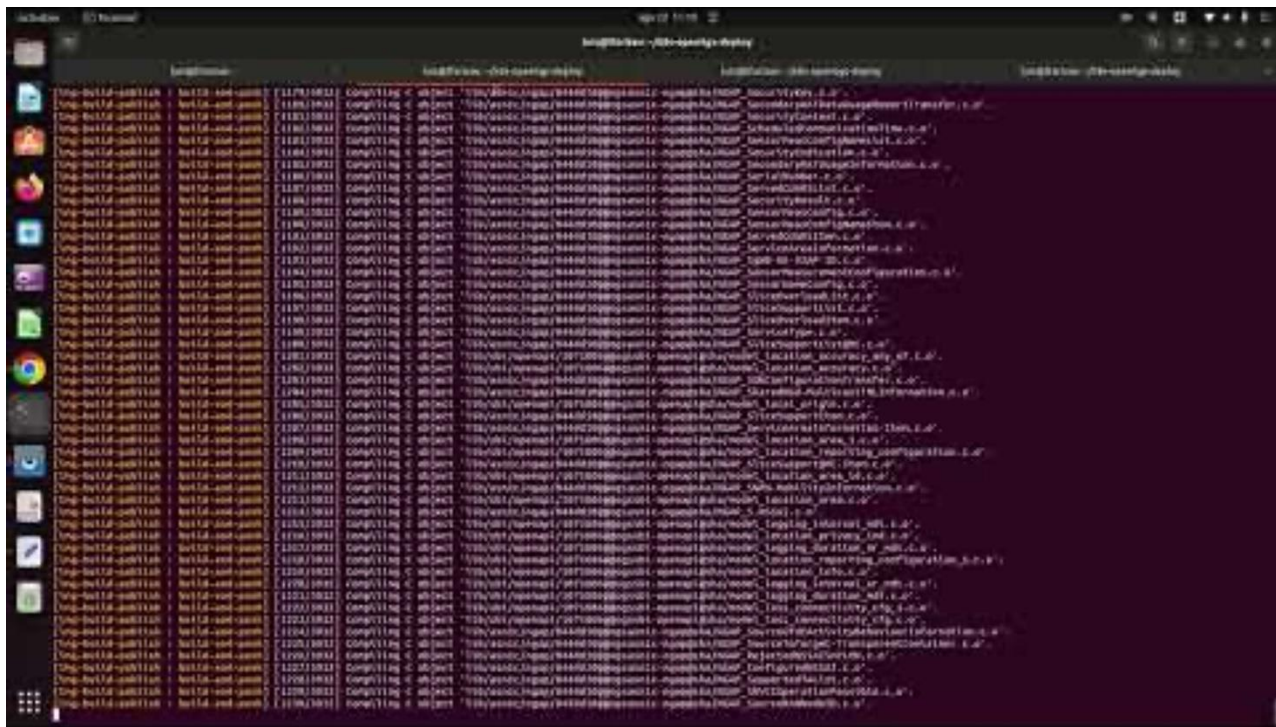
Image building allows us to generate images using the processor architecture and kernel (critical in cellular network deployments for RAN signal processing)

Step 4:
Tekton
(continuous integration)

```
luis@lfarizav:~$ tkn p ls
No Pipelines found
luis@lfarizav:~$ tkn pr ls
No PipelineRuns found
luis@lfarizav:~$ k apply -f /home/luis/k8s-open5gs/Tekton/Pipelines/k8s-open5gs/k8s_open5gs-ci-pipeline.yaml
pipeline.tekton.dev/k8s-open5gs-ci created
luis@lfarizav:~$ tkn p ls
NAME                AGE      LAST RUN      STARTED      DURATION      STATUS
k8s-open5gs-ci      2 seconds ago  ---          ---          ---          ---
luis@lfarizav:~$ k create -f /home/luis/k8s-open5gs/Tekton/Pipelines/k8s-open5gs/k8s_open5gs-ci-pipelinerun.yaml
pipelinerun.tekton.dev/k8s-open5gs-pipeline-run-cip9wrj created
luis@lfarizav:~$ tkn pr ls
NAME                STARTED      DURATION      STATUS
k8s-open5gs-pipeline-run-cip9wrj  2 seconds ago  ---          Running
luis@lfarizav:~$ tkn pr ls
NAME                STARTED      DURATION      STATUS
k8s-open5gs-pipeline-run-cip9wrj  22 minutes ago  5m56s        Succeeded
luis@lfarizav:~$
```



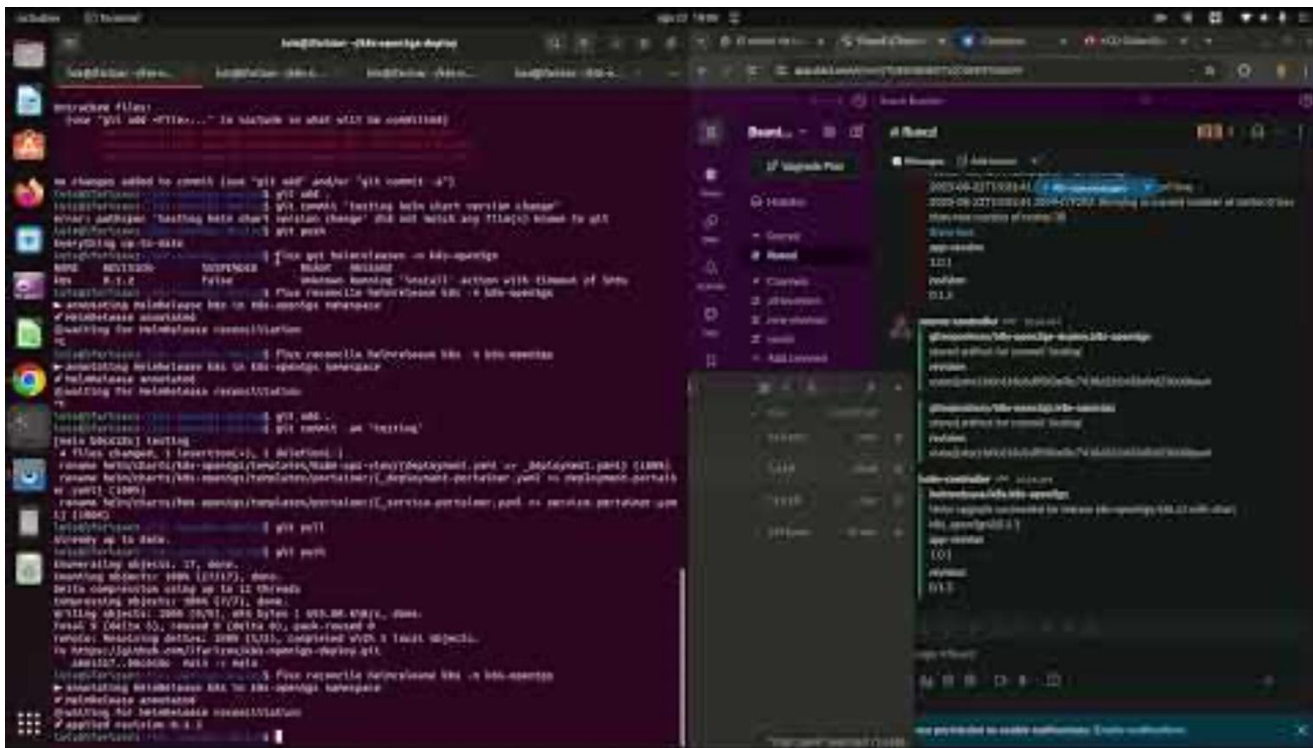
5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enabling CI/CD



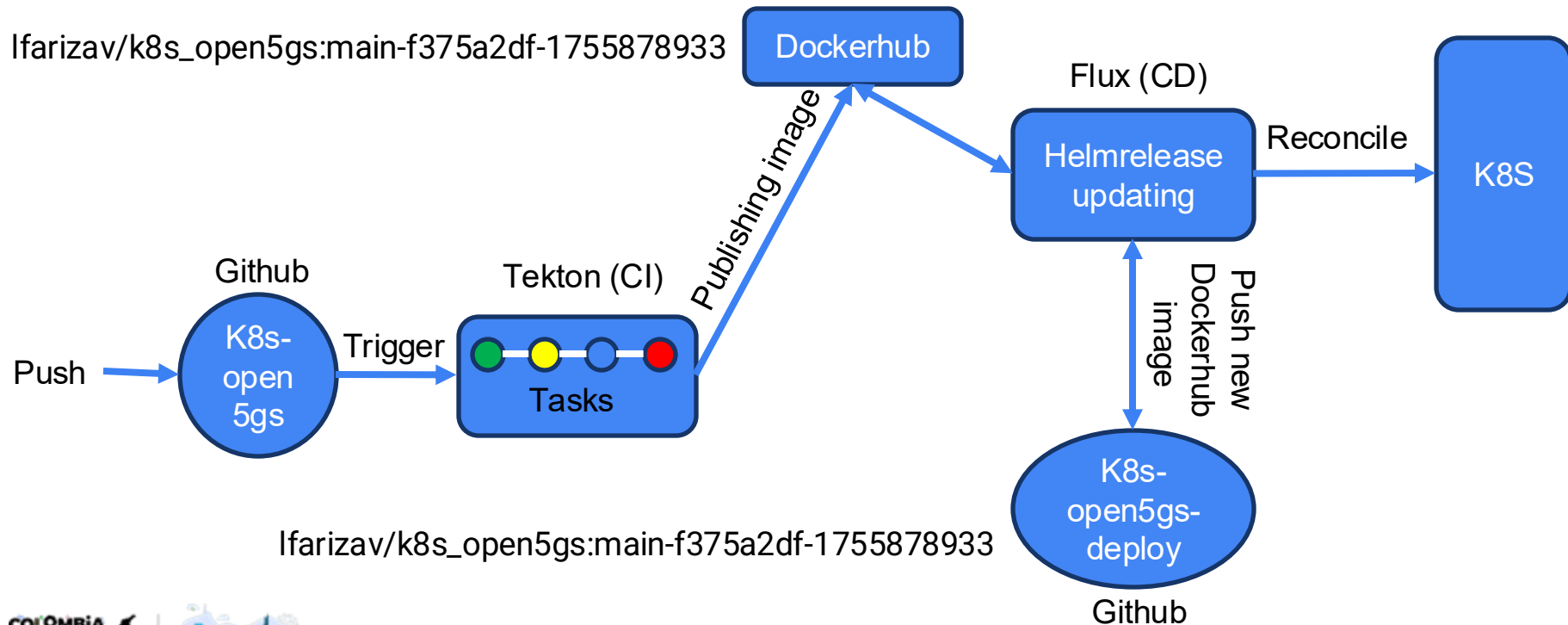
Step 5:
Testing
Tekton
(continuous
integration)

5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enabling CI/CD

Step 6: Testing Flux (Continuous Delivery)



5. Using KIND, Multus, Tekton (Kaniko+git-clone tasks), and FluxCD for deploying Kubernetes and enable CI/CD



6. Migrating from docker compose to Helm

 Grok

 Gemini

 Claude

 ChatGPT

 deepseek

Free versions

Partial results

Waste of time using free version

They can not manage such a big project

Do not consider networking issues

Do not consider security issues

Do not consider container migrated issues

Do not recommend how to build Kubernetes

6. Migrating from docker compose to Helm

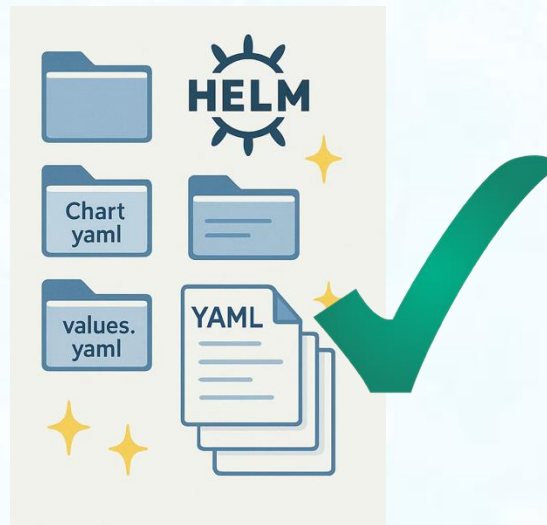
Automatic fast migration
Generates many yaml files
Manual install and uninstall
Do not follow the installation order
2 seconds

Fast solution



Best slow solution

Manually migration from scratch
Organized chart
Easy to install, upgrade and uninstall
Days or months



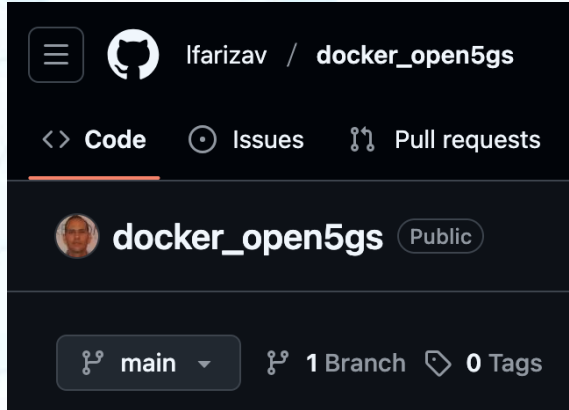
6. Migrating from docker compose to Helm

Main issues to overcome:

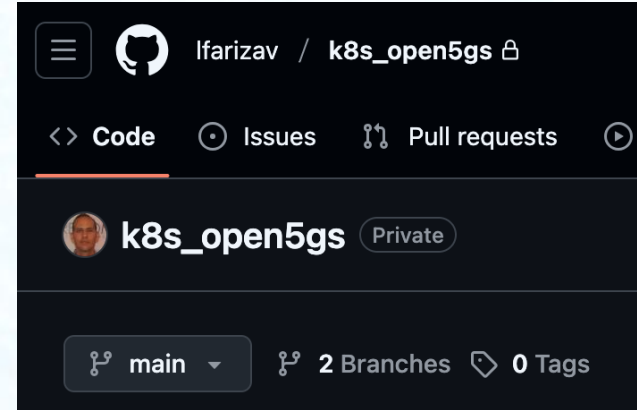
1. Management of volumes. It was necessary to create a cluster with 1 control-plane and 1 node.
2. Connectivity worked with services, however never a UE was attached neither get internet access. First solution include Multus but manage secondary ip with pods.
3. CI-CD was achieved partially. If I run both at the same time the computer resources were not enough.
4. Find the folder in /var/lib/docker to copy configuration files.
5. Stability and selfhealing request more processing. Kubernetes demands more than simple containers.



6. Migrating from docker compose to Helm (LTE)



https://github.com/herlesupreeth/docker_open5gs
https://github.com/MAlexVR/docker_open5gs



https://github.com/lfarizav/k8s_open5gs

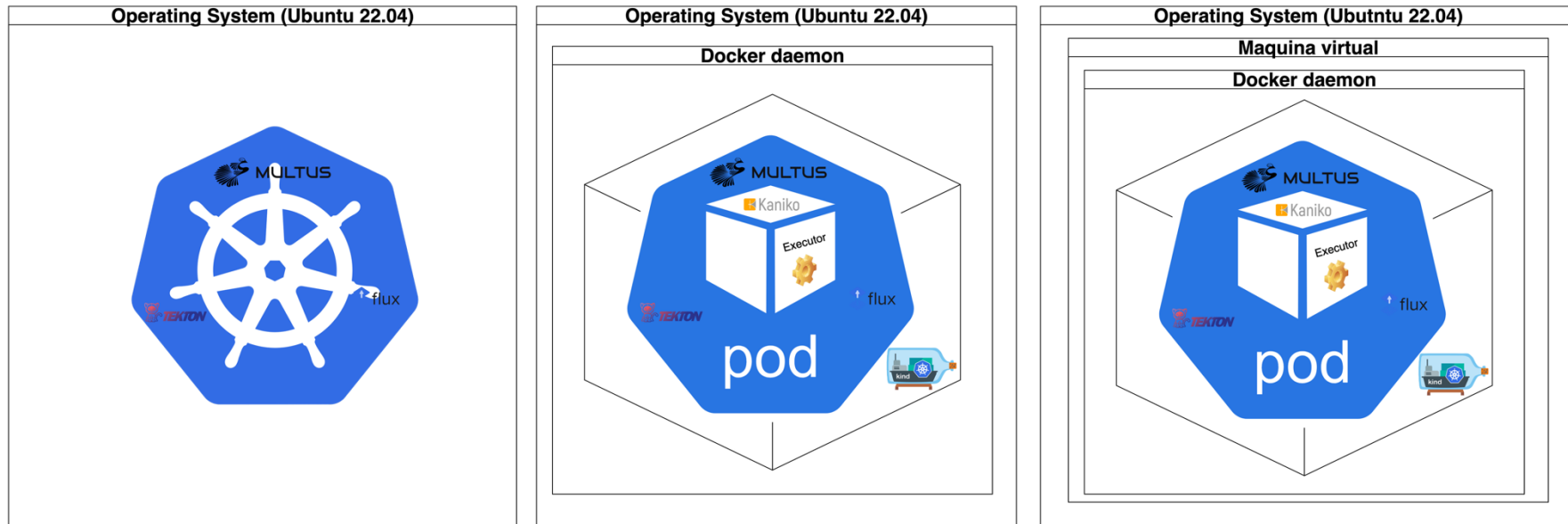


kubernetes



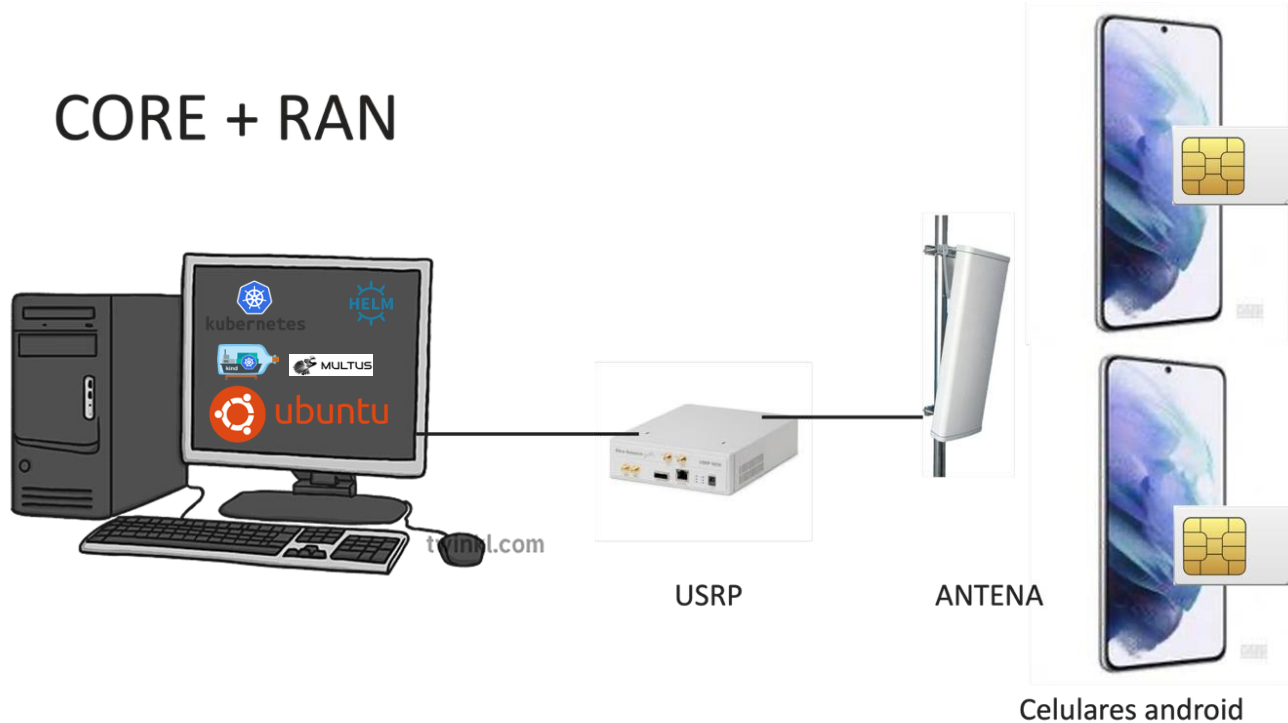
6. Migrating from docker compose to Helm (LTE)

Virtualization levels



6. Migrating from docker compose to Helm (LTE)

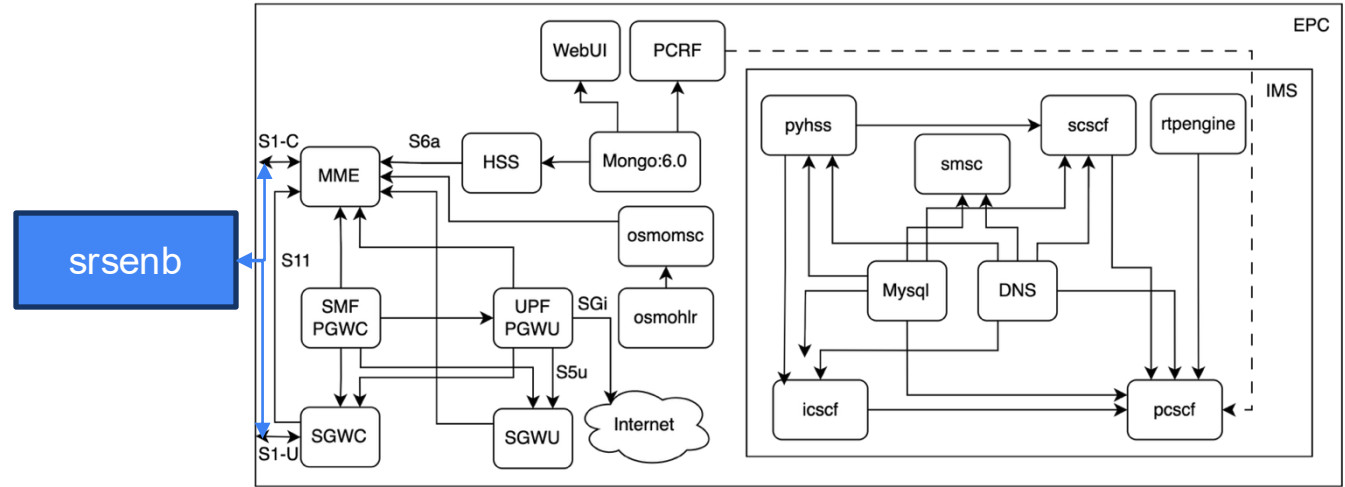
CORE + RAN



6. Migrating from docker compose to Helm (LTE)



kubernetes



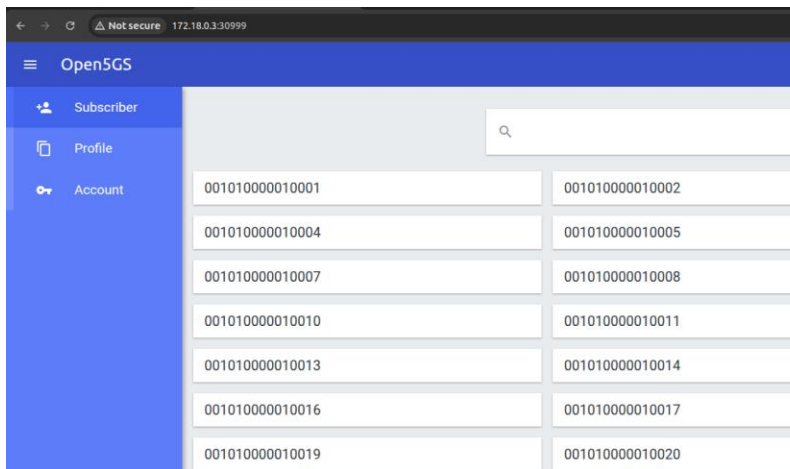
6. Migrating from docker compose to Helm (LTE)

```
luis@lfarizav:~$ watch kubectl get pods -n k8s-open5gs -o wide
luis@lfarizav:~$ watch kubectl get pods -n k8s-open5gs -o wide
luis@lfarizav:~$ kubectl get pods -n k8s-open5gs -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
k8s-dns-c566657bb-5q5b9	1/1	Running	0	14m	10.244.1.20	dev-worker	<none>	<none>
k8s-hss-7dbd666dfd-776jv	1/1	Running	0	14m	10.244.1.15	dev-worker	<none>	<none>
k8s-icscf-f4f9fcc94-8qsfh	1/1	Running	0	14m	10.244.1.3	dev-worker	<none>	<none>
k8s-mme-7684bdd65c-ldd7t	1/1	Running	0	14m	10.244.1.18	dev-worker	<none>	<none>
k8s-mongo-6894497489-qlpvz	1/1	Running	0	14m	10.244.1.10	dev-worker	<none>	<none>
k8s-mysql-5cb68b7885-dxcmc	1/1	Running	0	14m	10.244.1.5	dev-worker	<none>	<none>
k8s-osmohlr-5896bf9454-g9vkr	1/1	Running	0	14m	10.244.1.4	dev-worker	<none>	<none>
k8s-osmomsc-866cf6c96c-4p8xn	1/1	Running	0	14m	10.244.1.6	dev-worker	<none>	<none>
k8s-pcrf-5568547bff-7l4xb	1/1	Running	0	14m	10.244.1.11	dev-worker	<none>	<none>
k8s-pcscf-d59565f4f-8s579	1/1	Running	0	14m	10.244.1.22	dev-worker	<none>	<none>
k8s-pyhss-8f4859d6d-5k9hd	1/1	Running	0	14m	10.244.1.19	dev-worker	<none>	<none>
k8s-rtpengine-f5596ccf-gf28n	1/1	Running	0	14m	10.244.1.12	dev-worker	<none>	<none>
k8s-scscf-86c6d58846-mkkzk	1/1	Running	0	14m	10.244.1.16	dev-worker	<none>	<none>
k8s-sgwc-6995b4c688-2n6qz	1/1	Running	0	14m	10.244.1.17	dev-worker	<none>	<none>
k8s-sgwu-5467c46789-mg8dc	1/1	Running	0	14m	10.244.1.21	dev-worker	<none>	<none>
k8s-smf-c666db84d-ppfpq	1/1	Running	0	14m	10.244.1.8	dev-worker	<none>	<none>
k8s-smsc-84db9cb86f-rlqnl	1/1	Running	0	14m	10.244.1.7	dev-worker	<none>	<none>
k8s-srslte-6c49657bb-z4h8s	1/1	Running	0	14m	10.244.1.14	dev-worker	<none>	<none>
k8s-upf-565f49db77-zvnr	1/1	Running	0	14m	10.244.1.9	dev-worker	<none>	<none>
k8s-webui-77f84cb56f-zscfn	1/1	Running	0	14m	10.244.1.13	dev-worker	<none>	<none>
k8s-wowza-66999979cd-56mtp	1/1	Running	0	14m	10.244.1.2	dev-worker	<none>	<none>

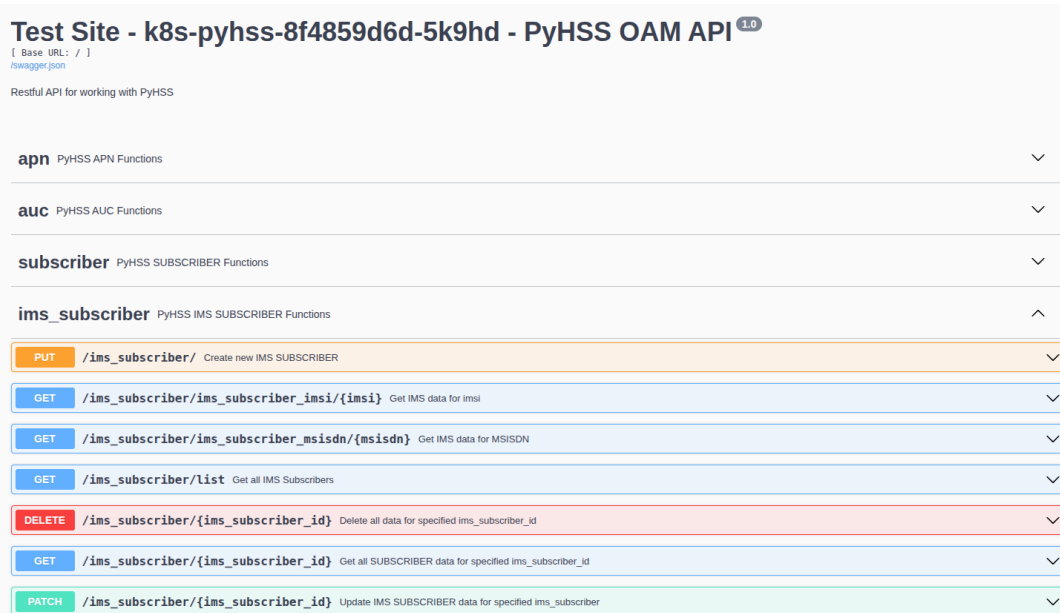
6. Migrating from docker compose to Helm (LTE)

6. Migrating from docker compose to Helm (LTE)



The screenshot shows the Open5GS web interface. On the left is a blue sidebar with a menu containing 'Subscriber', 'Profile', and 'Account'. The main area has a search bar and a table of subscribers. The table has two columns for IMSI numbers. The first column contains values from 001010000010001 to 001010000010019. The second column contains values from 001010000010002 to 001010000010020.

001010000010001	001010000010002
001010000010004	001010000010005
001010000010007	001010000010008
001010000010010	001010000010011
001010000010013	001010000010014
001010000010016	001010000010017
001010000010019	001010000010020



The screenshot shows the Swagger UI for the PyHSS OAM API. The title is 'Test Site - k8s-pyhss-8f4859d6d-5k9hd - PyHSS OAM API' with a version indicator '1.0'. Below the title, it says '[Base URL: /]' and 'swagger.json'. The description is 'Restful API for working with PyHSS'. There are four expandable sections: 'apn' (PyHSS APN Functions), 'auc' (PyHSS AUC Functions), 'subscriber' (PyHSS SUBSCRIBER Functions), and 'ims_subscriber' (PyHSS IMS SUBSCRIBER Functions). The 'ims_subscriber' section is expanded, showing several endpoints:

- PUT** `/ims_subscriber/` Create new IMS SUBSCRIBER
- GET** `/ims_subscriber/ims_subscriber_imsi/{imsi}` Get IMS data for imsi
- GET** `/ims_subscriber/ims_subscriber_msisdn/{msisdn}` Get IMS data for MSISDN
- GET** `/ims_subscriber/list` Get all IMS SUBSCRIBERS
- DELETE** `/ims_subscriber/{ims_subscriber_id}` Delete all data for specified ims_subscriber_id
- GET** `/ims_subscriber/{ims_subscriber_id}` Get all SUBSCRIBER data for specified ims_subscriber_id
- PATCH** `/ims_subscriber/{ims_subscriber_id}` Update IMS SUBSCRIBER data for specified ims_subscriber

6. Migrating from docker compose to Helm (LTE)

The screenshot displays the Lens Kubernetes dashboard interface. The main view shows a list of pods in the `k8s-open5gs` namespace. The pods are:

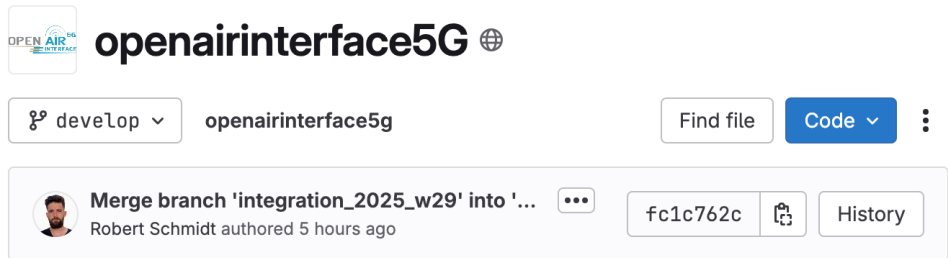
Name	Namespace	Containers	CPU	Memor	Restart	Controlled E	Node	QoS	Age	Status
k8s-dns-c566657bb-5q5b9	k8s-open5gs		N/A	N/A	0	ReplicaSet	dev-worke	BestEffort	2m52s	Running
k8s-mongo-6894497489-qlpvz	k8s-open5gs		N/A	N/A	0	ReplicaSet	dev-worke	BestEffort	2m53s	Running
k8s-mysql-5b69h7995-dvzmc	k8s-open5gs		N/A	N/A	0	ReplicaSet	dev-worke	BestEffort	2m53s	Running

Overlaid on the dashboard are three mobile status screens:

- Phone 0**: Shows general settings for the mobile network, including Network Type (NR/LTE), Signal Strength, and various service toggles.
- Phone info1**: Displays detailed information about the mobile network, including IMEI, Phone Number, and Network Type (NR/LTE).
- Información del teléfono**: Shows detailed information about the mobile network, including IMEI, Phone Number, and Network Type (NR/LTE).

6. Migrating from docker compose to Helm (5G-SA)

oai / openairinterface5G

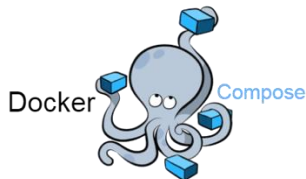


openairinterface5G

develop openairinterface5g Find file Code

Merge branch 'integration_2025_w29' into '... Robert Schmidt authored 5 hours ago fc1c762c History

git clone <https://gitlab.eurecom.fr/oai/openairinterface5g.git>
git clone <https://github.com/lfarizav/oai-cn5g.git>
https://github.com/OPENAIRINTERFACE/openairinterface5g/blob/develop/doc/NR_SA_Tutorial_OAI_CN5G.md



kubernetes

To be continued



7. Management of external ports (USB3.0)



volumeMounts:

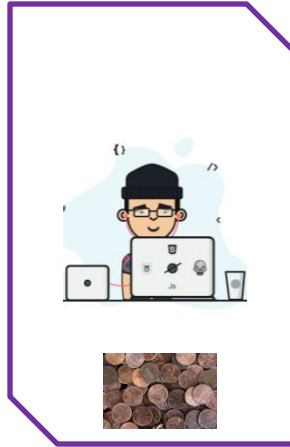
- name: srslte-serial
mountPath: /dev/serial
- name: srslte-usb
mountPath: /dev/bus/usb

volumes:

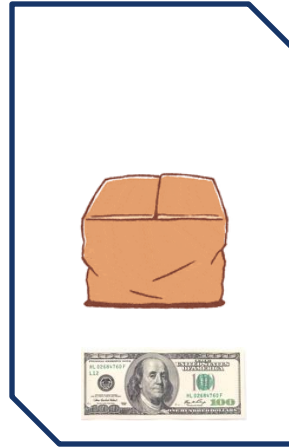
- name: srslte-serial
hostPath:
path: /dev/serial
- name: srslte-usb
hostPath:
path: /dev/bus/usb

8. Security issues

Proactive



Develop

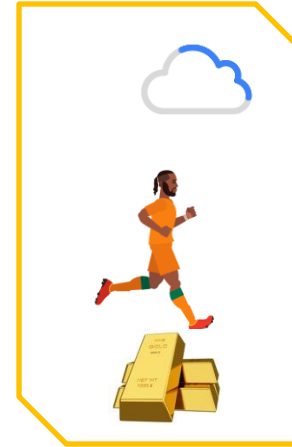


Distribute



Deploy

Reactive



Runtime

Loss of money
Security issues
discovery



8. Security issues

Access to /dev host folder.

Managing password through environment variables

Base image for Dockerfile build has plenty of vulnerabilities (Trivy)

Many images have curl and wget installed

There is not service mesh with mTLS installed, so containers communicates unencrypted

Secrets are unencrypted (only base64 encoded)



8. Security issues

```

luts@lforti:~$ curl -L https://github.com/aquasecurity/kube-bench/releases/download/v0.6.2/kube-bench_0.6.2_linux_amd64.tar.gz -o kube-bench_0.6.2_linux_amd64.tar.gz

tar -xvf kube-bench_0.6.2_linux_amd64.tar.gz
current
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
                                Dload  Upload  Total  Spent    Left  Sp
0         0  0     0    0     0      0  0 --:--:-- --:--:-- --:--:-- 
100 7821k 100 7821k  0     0  4429k    0  0:00:01 0:00:01 --:--:-- 2
0.4M
cfg/ack-1.0/config.yaml
cfg/ack-1.0/controlplane.yaml
cfg/ack-1.0/etcd.yaml
cfg/ack-1.0/managedservices.yaml
cfg/ack-1.0/master.yaml

```

```

[INFO] Farizav: $ ./kube-bench --nftig-dir 'pwd'/cfg --config 'pwd'/cfg/config.yaml
[INFO] 1 Master Node Security Configuration
[INFO] 1.1 Master Node Configuration Files
[FAIL] 1.1.1 Ensure that the API server pod specification file permissions are set to 644 or more restrictive (Automated)
[FAIL] 1.1.2 Ensure that the API server pod specification file ownership is set to root:root (Automated)
[FAIL] 1.1.3 Ensure that the controller manager pod specification file permissions are set to 644 or more restrictive (Automated)
[FAIL] 1.1.4 Ensure that the controller manager pod specification file ownership is set to root:root (Automated)
[FAIL] 1.1.5 Ensure that the scheduler pod specification file permissions are set to 644 or more restrictive (Automated)
[FAIL] 1.1.6 Ensure that the scheduler pod specification file ownership is set to root:root (Automated)
[FAIL] 1.1.7 Ensure that the etcd pod specification file permissions are set to 644 or more restrictive (Automated)
[FAIL] 1.1.8 Ensure that the etcd pod specification file ownership is set to root:root (Automated)
[WARN] 1.1.9 Ensure that the Container Network Interface file permissions are set to 644 or more restrictive (Manual)
[WARN] 1.1.10 Ensure that the Container Network Interface file ownership is set to root:root (Manual)
[FAIL] 1.1.11 Ensure that the etcd data directory permissions are set to 700 or more restrictive (Automated)
[FAIL] 1.1.12 Ensure that the etcd data directory ownership is set to etcd:etcd (Automated)
[FAIL] 1.1.13 Ensure that the admin.conf file permissions are set to 644 or more restrictive (Automated)
[FAIL] 1.1.14 Ensure that the admin.conf file ownership is set to root:root (Automated)
[FAIL] 1.1.15 Ensure that the scheduler.conf file permissions are set to 644 or more restrictive (Automated)
[FAIL] 1.1.16 Ensure that the scheduler.conf file ownership is set to root:root (Automated)
[FAIL] 1.1.17 Ensure that the controller-manager.conf file permissions are set to 644 or more restrictive (Automated)
[FAIL] 1.1.18 Ensure that the controller-manager.conf file ownership is set to root:root (Automated)
[FAIL] 1.1.19 Ensure that the Kubernetes PKI directory and file ownership is set to root:root (Automated)
[WARN] 1.1.20 Ensure that the Kubernetes PKI certificate file permissions are set to 644 or more restrictive (Manual)
[WARN] 1.1.21 Ensure that the Kubernetes PKI key file permissions are set to 600 (Manual)
[INFO] 1.2 API Server
[WARN] 1.2.1 Ensure that the --anonymous-auth argument is set to false (Manual)
[PASS] 1.2.2 Ensure that the --basic-auth-file argument is not set (Automated)
[PASS] 1.2.3 Ensure that the --token-auth-file parameter is not set (Automated)
[PASS] 1.2.4 Ensure that the --kubelet-https argument is set to true (Automated)
[PASS] 1.2.5 Ensure that the --kubelet-client-certificate and --kubelet-client-key arguments are set as appropriate (Automated)
[FAIL] 1.2.6 Ensure that the --kubelet-certificate-authority argument is set as appropriate (Automated)
[PASS] 1.2.7 Ensure that the --authorization-mode argument is not set to AlwaysAllow (Automated)
[PASS] 1.2.8 Ensure that the --authorization-mode argument includes Node (Automated)
[PASS] 1.2.9 Ensure that the --authorization-mode argument includes RBAC (Automated)
[WARN] 1.2.10 Ensure that the admission control plugin EventRateLimit is set (Manual)
[PASS] 1.2.11 Ensure that the admission control plugin AlwaysAdmit is not set (Automated)
[WARN] 1.2.12 Ensure that the admission control plugin AlwaysPullImages is set (Manual)
[WARN] 1.2.13 Ensure that the admission control plugin SecurityContextDeny is set if PodSecurityPolicy is not used (Manual)
[PASS] 1.2.14 Ensure that the admission control plugin ServiceAccount is set (Automated)
[PASS] 1.2.15 Ensure that the admission control plugin NamespaceLifecycle is set (Automated)
[FAIL] 1.2.16 Ensure that the admission control plugin PodSecurityPolicy is set (Automated)
[PASS] 1.2.17 Ensure that the admission control plugin NodeRestriction is set (Automated)
[PASS] 1.2.18 Ensure that the --insecure-bind-address argument is not set (Automated)
[FAIL] 1.2.19 Ensure that the --insecure-port argument is set to 0 (Automated)
[PASS] 1.2.20 Ensure that the --secure-port argument is not set to 0 (Automated)
[FAIL] 1.2.21 Ensure that the --profiling argument is set to false (Automated)
[FAIL] 1.2.22 Ensure that the --audit-log-path argument is set (Automated)
[FAIL] 1.2.23 Ensure that the --audit-log-maxage argument is set to 30 or as appropriate (Automated)
[FAIL] 1.2.24 Ensure that the --audit-log-maxbackup argument is set to 10 or as appropriate (Automated)
[FAIL] 1.2.25 Ensure that the --audit-log-maxsize argument is set to 100 or as appropriate (Automated)

```

Kube-bench implements the CIS benchmark as closely as possible

9. Benchmark Docker compose vs Helm capabilities



9. Benchmark Docker compose vs Helm capabilities



1. Easy install/uninstall using docker compose up and down
2. Manual process to configure yaml files
3. Fast deploying process of containers
4. If a container stops, the application stops
5. Easy to bind an specific network from AMF
6. CI/CD tools can be used but installing on the host machine



1. Easy install/uninstall using helm install/upgrade/uninstall
2. Organized process to configure yaml files with values.yaml
3. Slow deploying process of pods
4. If pod stops, the deployment and replicaset starts the pod again
5. It is necessary to create a shared volume if we have more than 1 worked node
6. We can easily implement network policies
7. It was necessary to install Multus on Kubernetes to enable binding networks from AMF
8. It was enable CI/CD using CRDs of Flux and Tekton
9. It is possible to enable RBAC restrictions

10. Conclusions and takeaways

1. Before start a project it is necessary to study and build fundational bases on tools.
 2. The first think to create a project is to buil its architecture.
 3. Best practices and security compliance are a must.
 4. Focusing on details and troubleshooting are excelent habilities to build during a project.
 5. Enhancing Stability and Enabling Cloud-Native Capabilities for Private Cellular Networks are important issues for mobile network operators.
 6. A Kubernetes implementation is not the best option for all companies with different problems.
 7. Open-source tools can have some bugs; the idea is to overcome them and contribute.
 8. There is not shortcuts, a good project needs planning, strategy and goals in a determined time.
 9. LLMs can help you to solve small issues if you ask with plenty of details. When the complexity appears, now is not an option.
 10. Community is needed to build to help others to solve common problems and share knowledge.
 11. Invest time in fostering scripting to reduce repetitive common tasks.
 12. We can use kustomize to deploy and reconcile Kubernetes deployments.
 13. Using FluxCD and Tekton allows to use CRDs without instaling tools.
- If you delete a namespace you delete helmrelease.



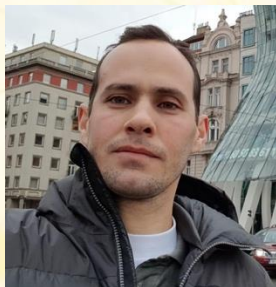


QUESTIONS

OpenInfra
USER GROUP



COLOMBIA



Luis Felipe Ariza Vesga
Colombia OpenInfra User Group Organizer
lfarizav@gmail.com

